3. Transformer architecture for Large Language Models

CSC3043S Reasoning in AI: Large Language Models

Dr Jan Buys

University of Cape Town

Slides from Tatsu Hashimoto (Stanford CS336), others where cited



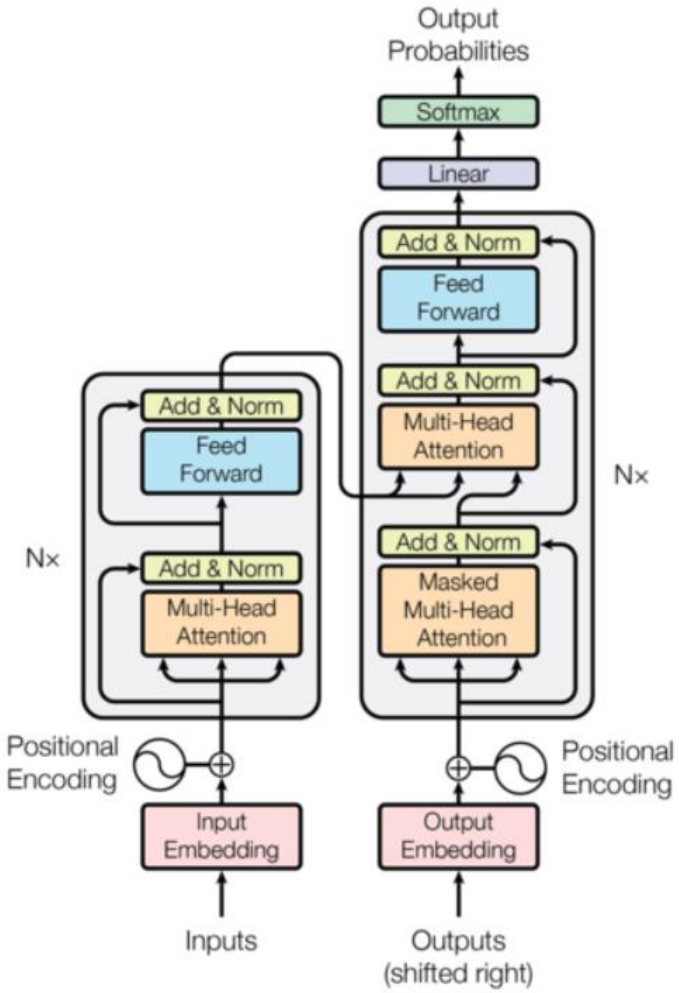
UNIVERSITY OF CAPE TOWN

IYUNIVESITHI YASEKAPA • UNIVERSITEIT VAN KAAPSTAD

Transformers

- The Transformer is the dominant architecture for Large Language Models
- Since the original Transformer paper there has been a lot of research into how to improve the performance and scalability of the Transformer for training LLMs
- There isn’t a single standard version of the architecture, but several architectural choices have become standard that are different than in the original Transformer

Transformer architecture

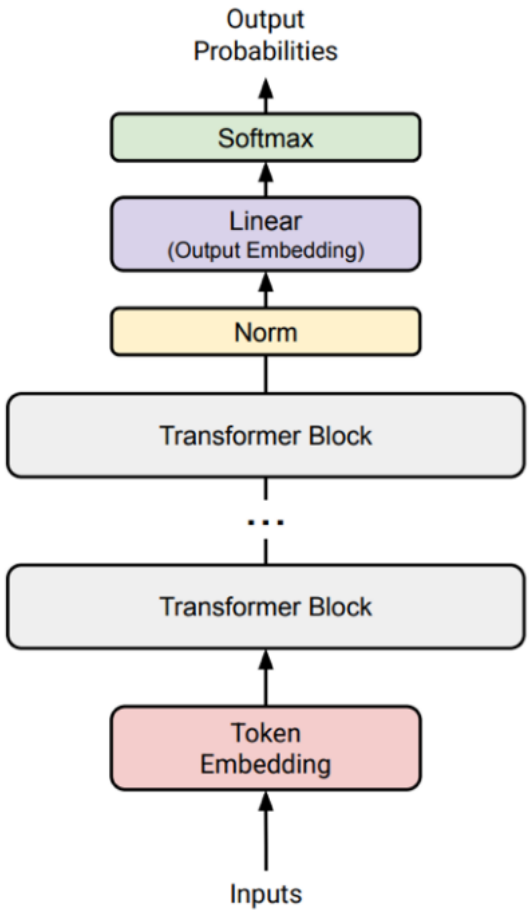


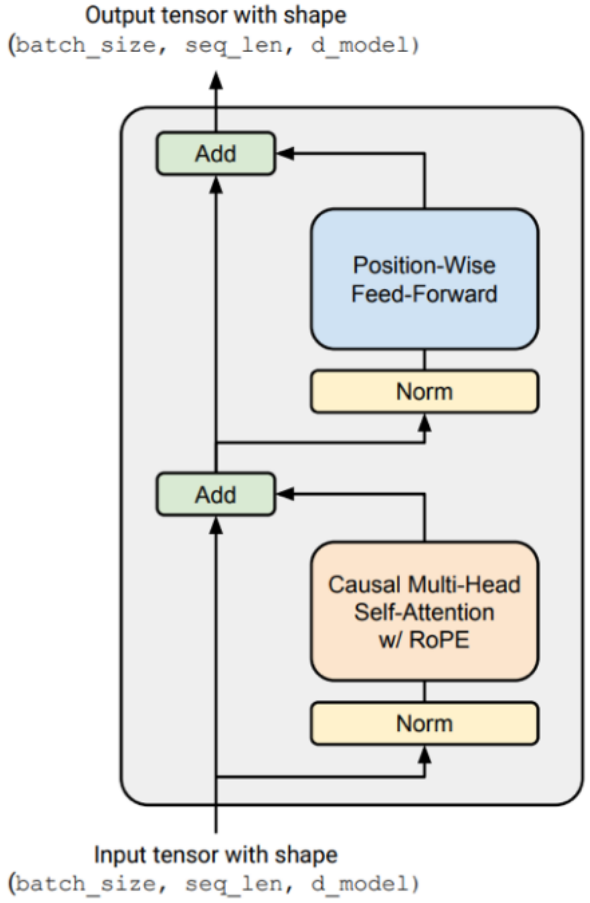
Vaswani et al. (2017)

Original architecture:

- Stacked layers with multi-head attention (MHA) and feed forward networks (FFN) per position
- Positional embeddings based on sines and cosines
- LayerNorm applied after attention and feed forward layers, together with residual connections
- ReLU activation function in FFN

Transformer architecture





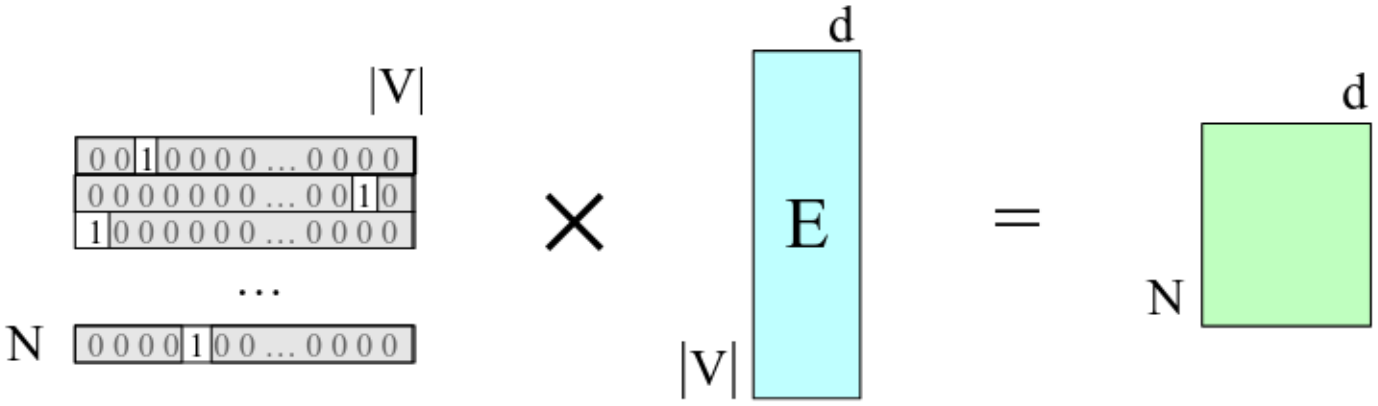
Modern variant:

- Grouped-Query Attention and KV cache for MHA
- Rotary position embeddings (RoPE)
- FF layers using SwiGLU, no bias terms
- LayerNorm (RMSNorm) before MHA/FF blocks

Plus particular hyperparameter choices

Token embedding

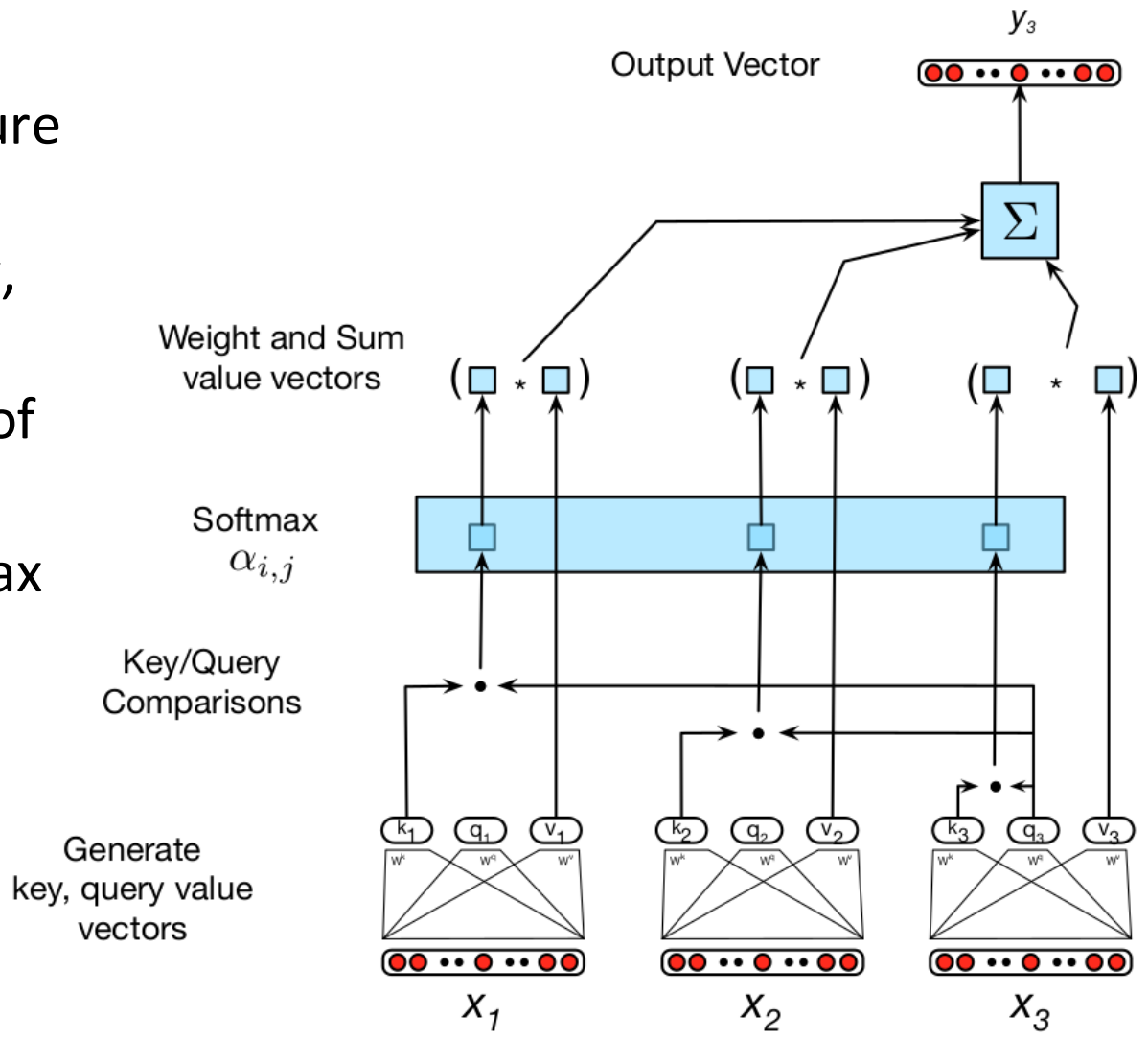
- After tokenization, each token is mapped to an input token embedding
- The embeddings vectors are stored in an embedding matrix **E** with dimension $|\mathrm{V}| \times d$ for vocabulary V
- The embedding lookup is implemented by representing each token index with a one-hot vector and multiplying with **E**



Jurafsky and Martin (2026) Ch. 6

Self-attention

- The core component of the architecture is self-attention
- Each input vector x_i is mapped to key, query and value vectors
- Query vector q_i is compared to each of the key vectors k_j in the sequence
- These scores are fed through a softmax and used to compute a weighted average of the values vectors v_j

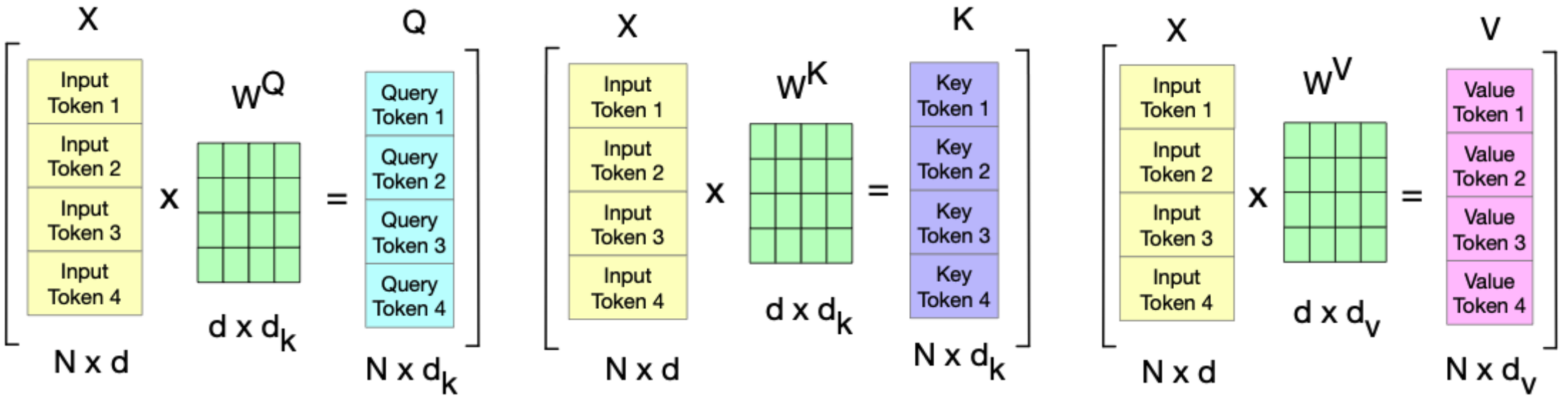


Jurafsky and Martin (2020) Ch. 9

Self-attention

Self-attention can be computed across time steps through matrix multiplication. Using a single matrix *X* for the input vectors:

$$Q = W^Q X;\ \ K = W^K X;\ \ V = W^V X$$

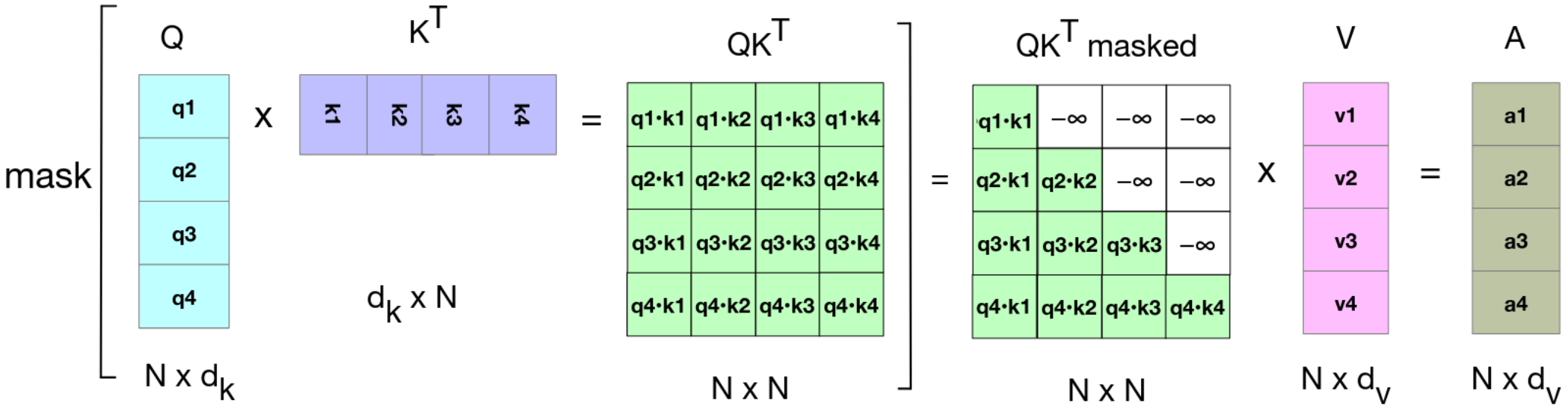


Jurafsky and Martin (2026) Ch. 8

Self-attention

Self-attention can be computed across time steps through matrix multiplication. In a decoder model the attention is masked

$$SelfAttention(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Jurafsky and Martin (2026) Ch. 8

Multi-head self-attention

Capture multiple relations between the elements by computing multiple attention scores, each with its own set of parameters

$$W_i^K,\ W_i^Q \text{ and } W_i^V$$

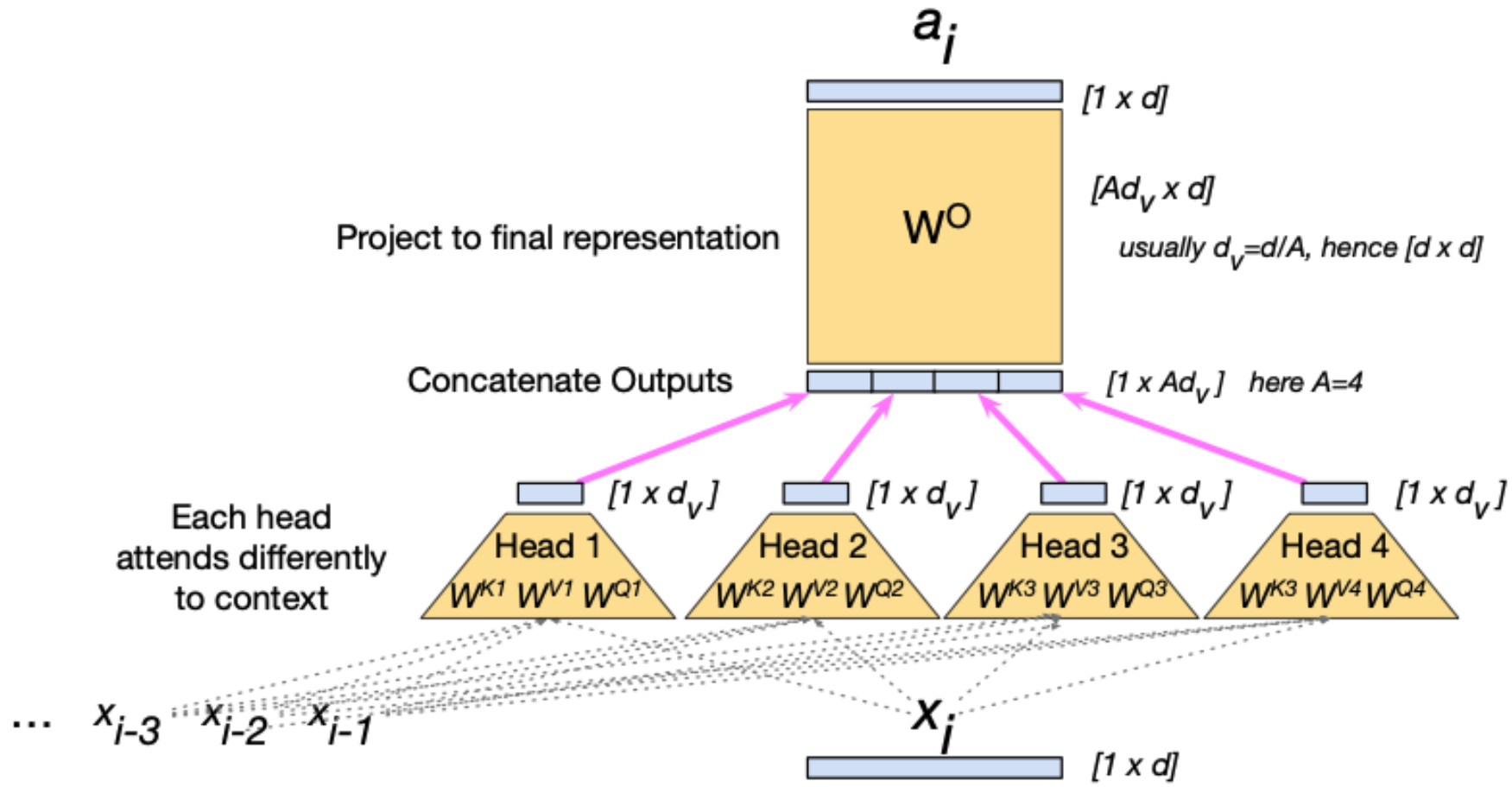
- Each attention computation is called an *attention head*

$$head_i = SelfAttention(W_i^Q X, W_i^K X, W_i^V X)$$

- In the final representation the attention heads are concatenated, followed by another linear projection

$$MultiHeadAttn(Q,K,V) = W^O(head_1 \oplus head_2 \ldots \oplus head_h)$$

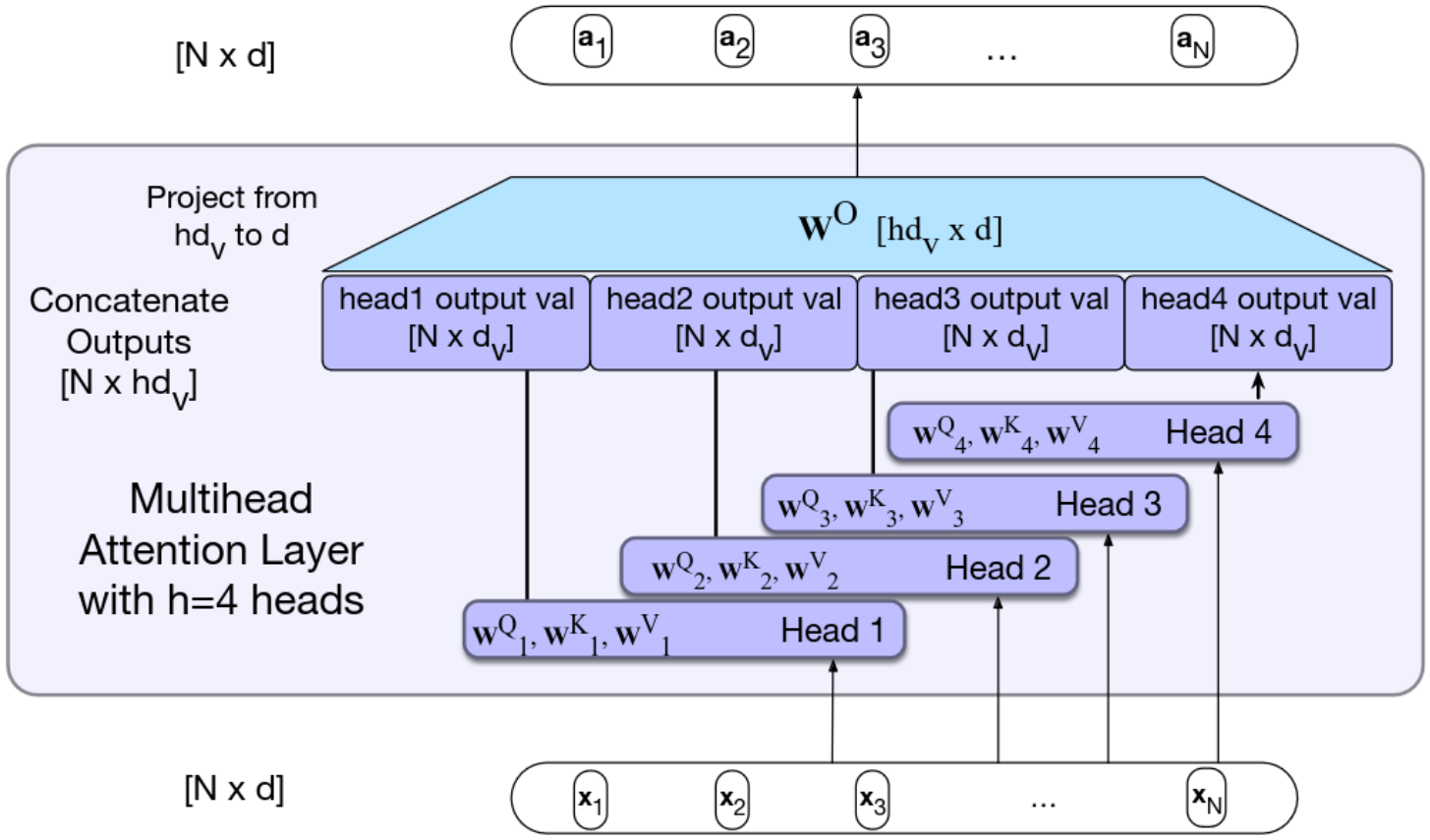
Multi-head self-attention



Jurafsky and Martin (2026) Ch. 8

Multi-head self-attention

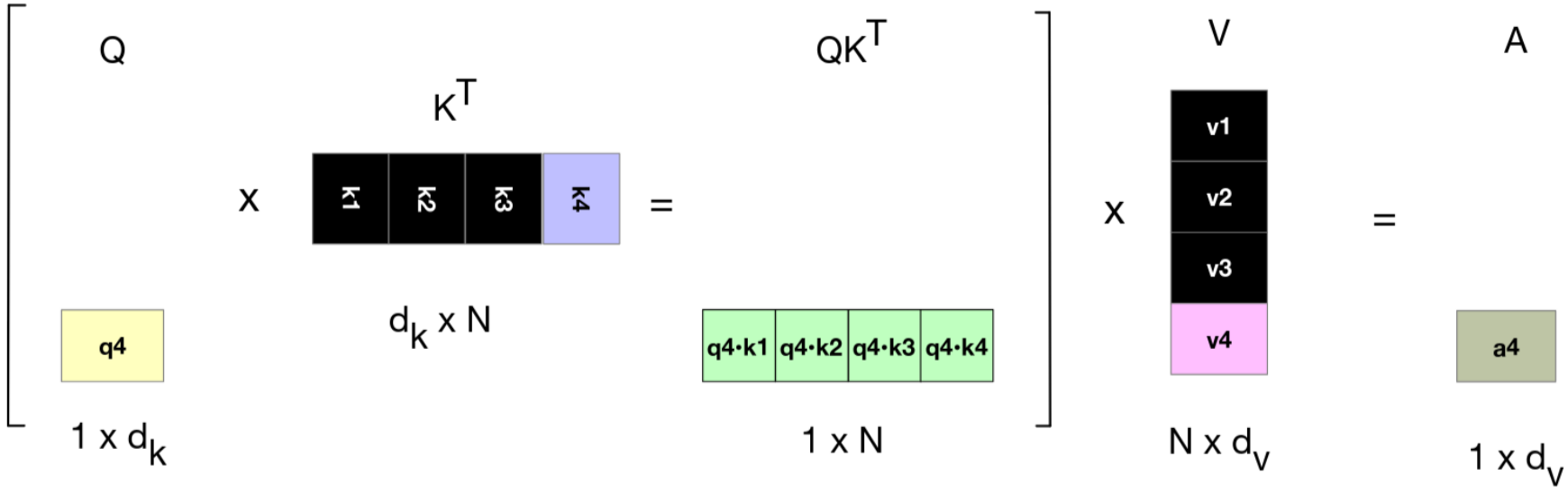
- Set the dimensions as: $d_q = d_v = d_k = \frac{d}{h}$
- Total number of parameters W and computation stay similar to single head



Jurafsky and Martin (2024) Ch. 10

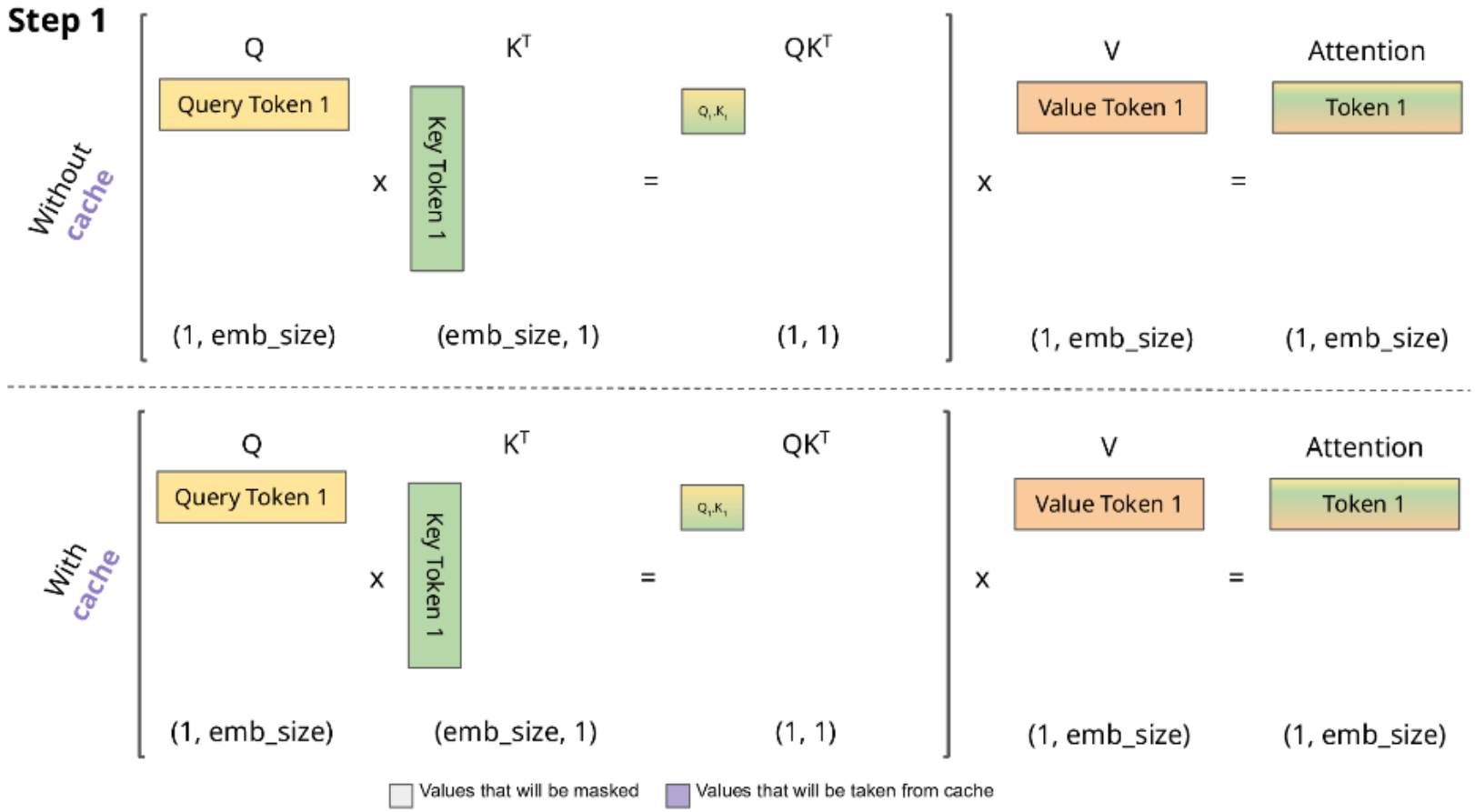
Self-attention KV cache

- During inference (generating text), the context size increases by 1 at each time step, so attention has to be recomputed
- But we can make this more efficient by cashing keys and values to be reused across time steps



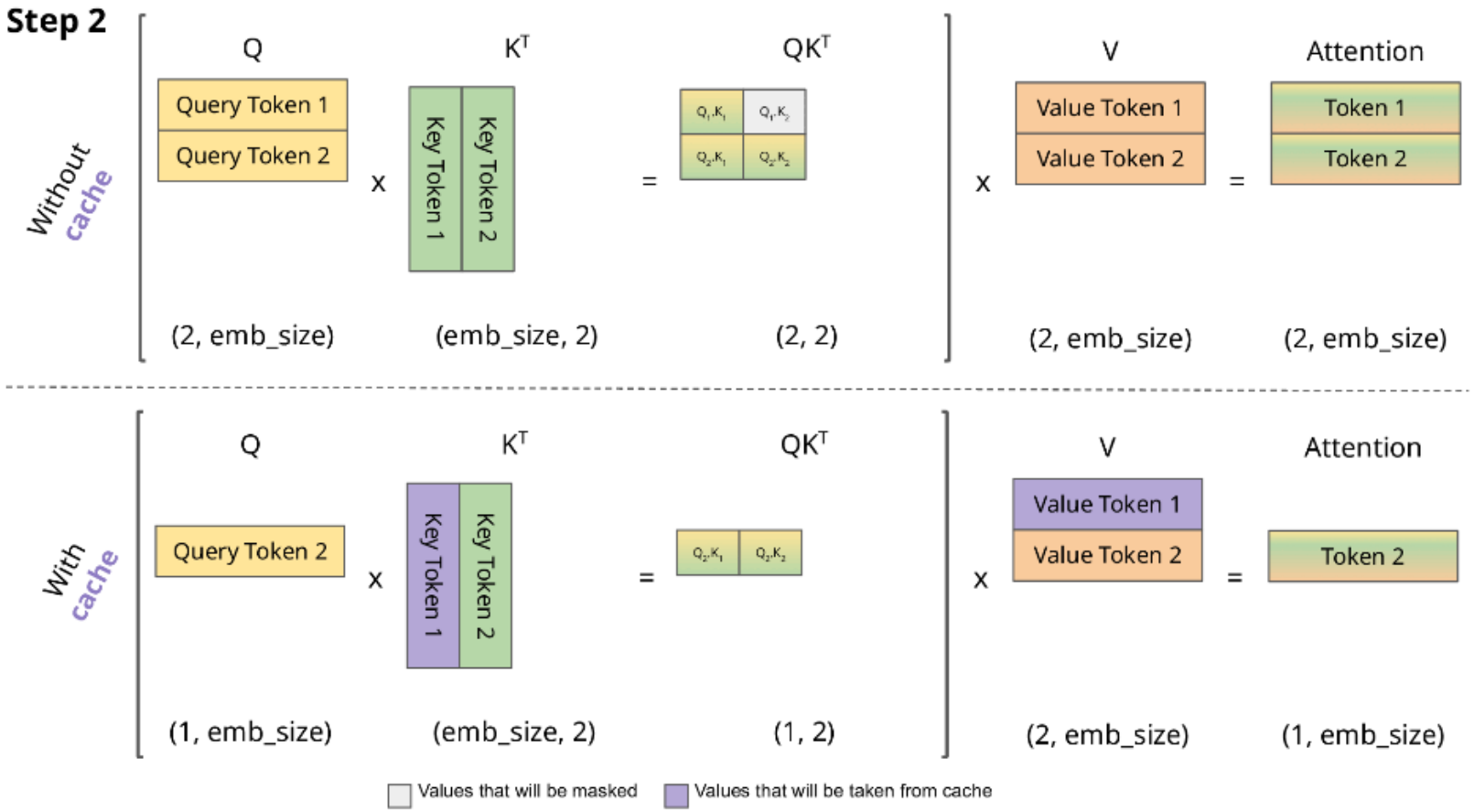
Jurafsky and Martin (2026) Ch. 8

Self-attention KV cache



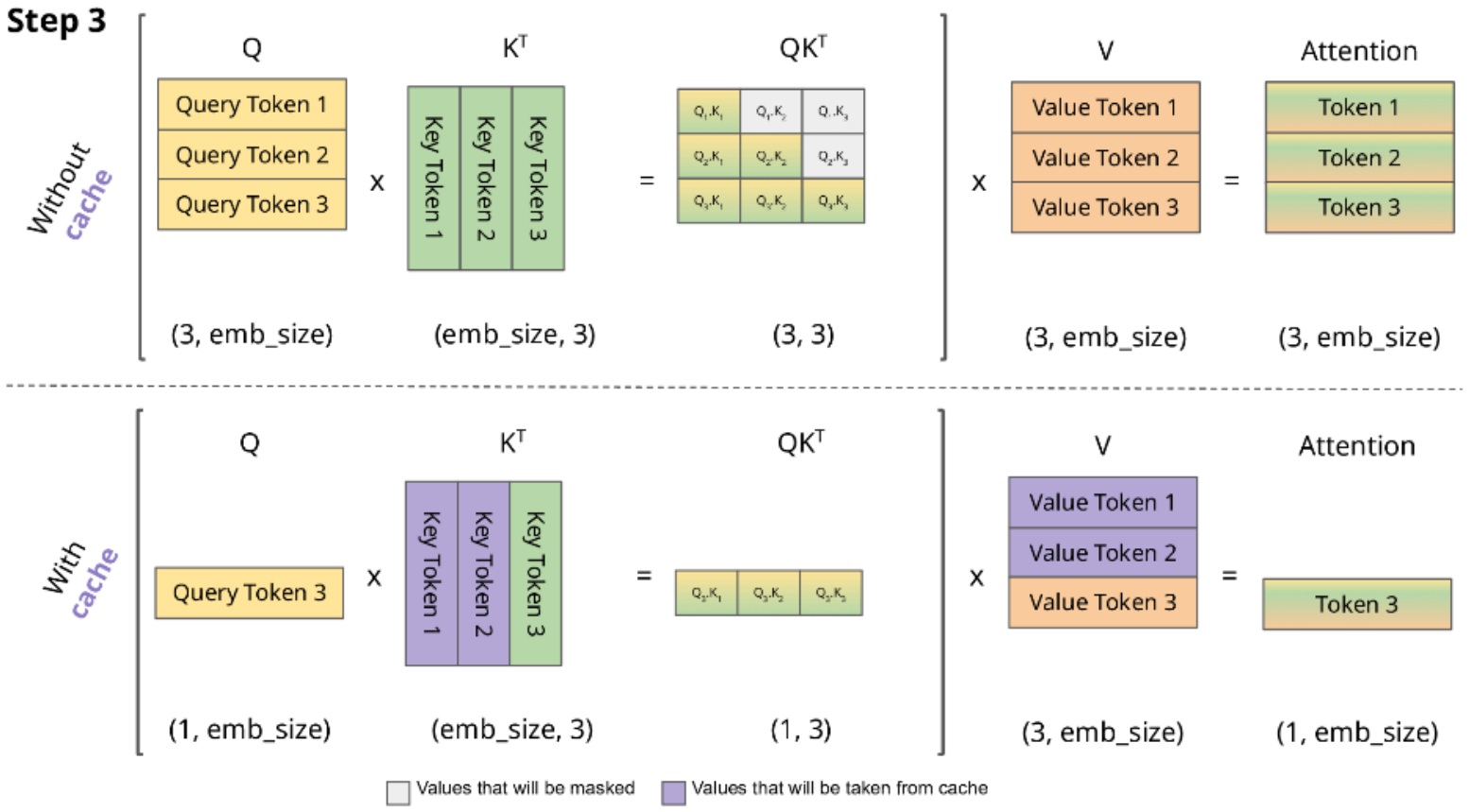
https://medium.com/@joaolages/kv-caching-explained-276520203249

Self-attention KV cache



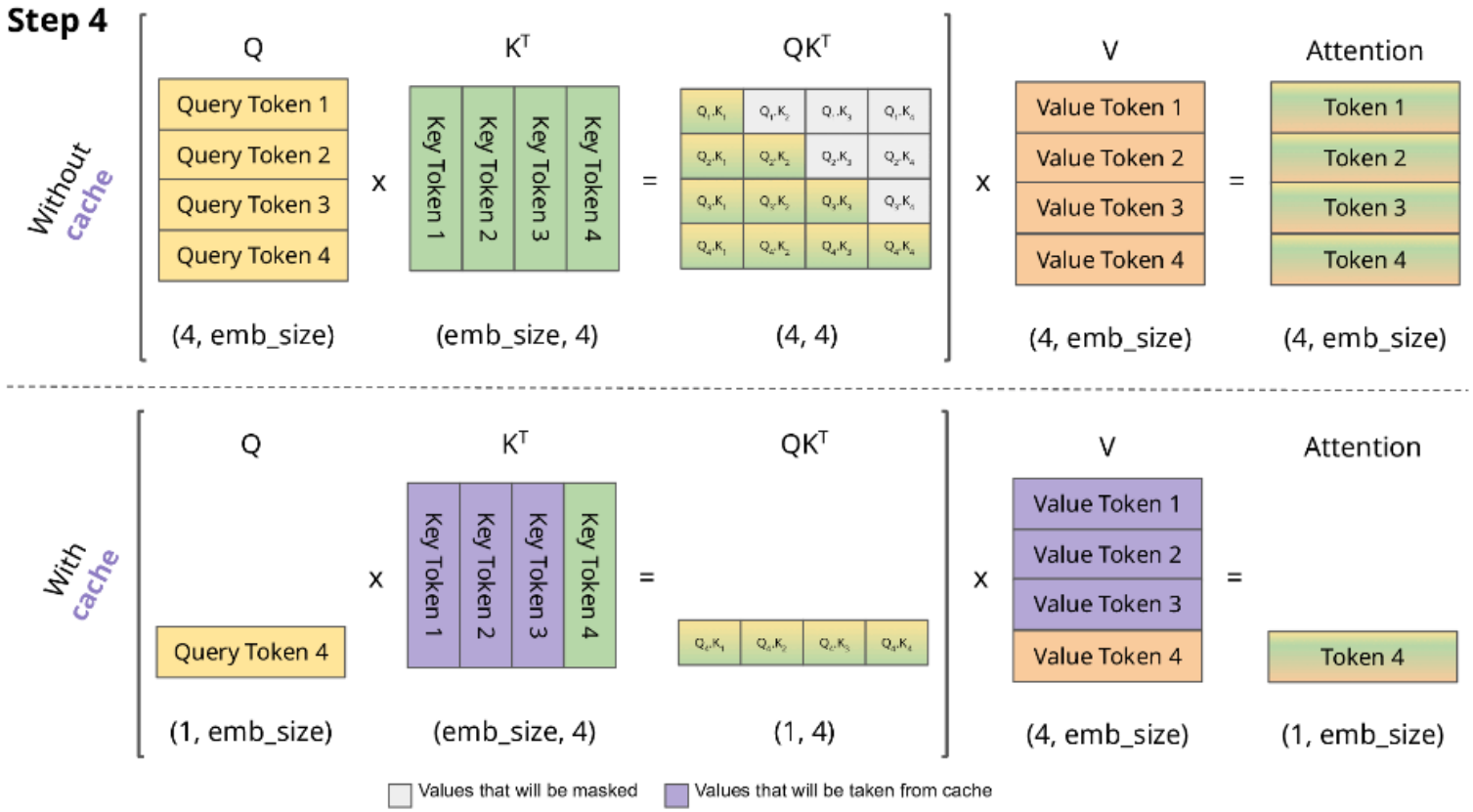
https://medium.com/@joaolages/kv-caching-explained-276520203249

Self-attention KV cache



https://medium.com/@joaolages/kv-caching-explained-276520203249

Self-attention KV cache



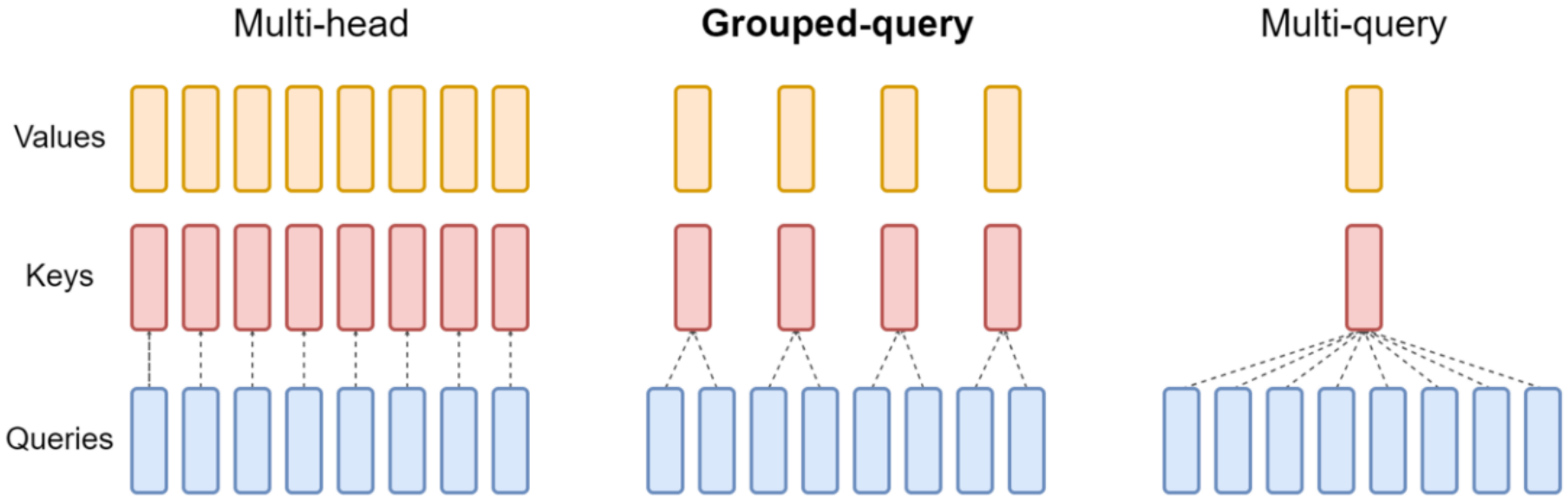
https://medium.com/@joaolages/kv-caching-explained-276520203249

Grouped-Query Attention (GQA)

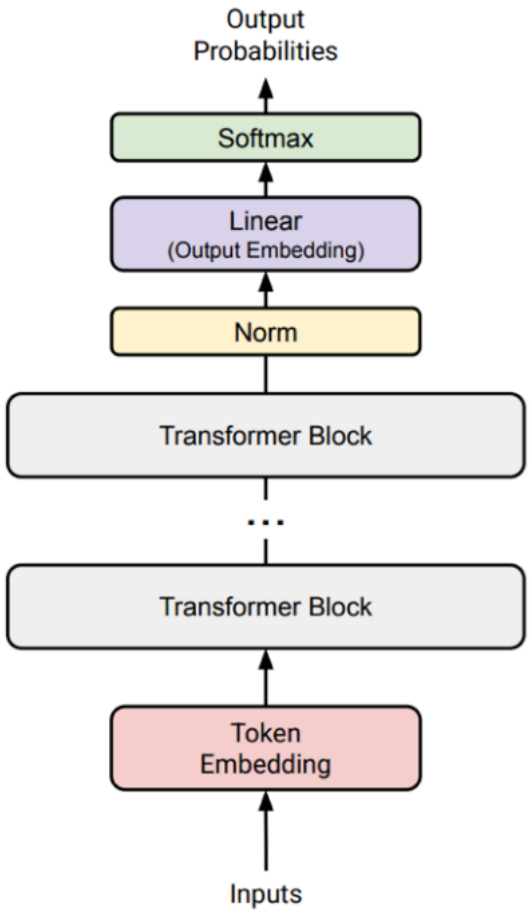
- Using a KV cache requires more memory operations which constrains computational efficiency
- This can be reduced by sharing keys and values across attention heads (at a given position – i.e., only 1 key/value parameter matrix)
- Multi-query attention (MQA): single key and value head (queries need to be separate per head)
- Grouped-query attention (GQA) – have multiple key/value heads, but each shared across a group of multiple queries
- Fewer key/value heads need to be accessed from the KV cache

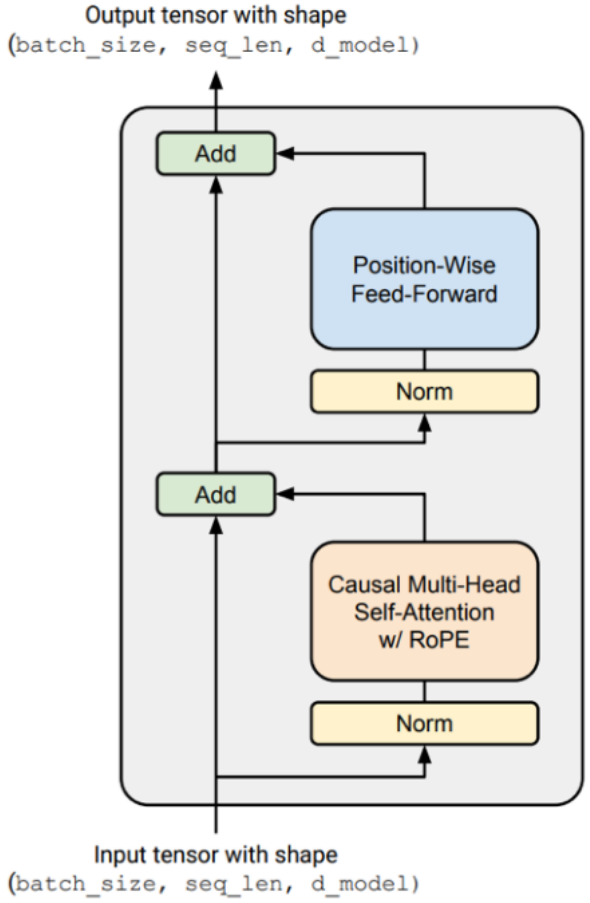
Grouped-Query Attention (GQA)

- Key-query ratio controls expressiveness
- Possible small performance decrease (and fewer key/value parameters for given dimensionality), but increases inference speed



Transformer architecture





Modern variant:

- Grouped-Query Attention and KV cache for MHA
- **Rotary position embeddings (RoPE)**
- FF layers using SwiGLU, no bias terms
- LayerNorm (RMSNorm) before MHA/FF blocks

Plus particular hyperparameter choices

Positional embeddings

- Neither the attention mechanism nor any other component in the Transformer architecture encodes the position of input vectors
- Various types of positional embeddings have been proposed:
  - Fixed sine/cosine-based embedding formula (original transformer)
  - Absolute embeddings: learn a positional embedding vector for each position (early LLMs)
  - Relative embeddings: add a vector to the attention computation encoding the relative positional distance between the key and query
- Most current LLMs use a variant of relative embeddings called RoPE

RoPE: rotary position embeddings

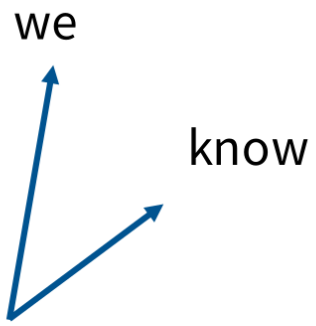
- Applies a rotation to input vectors using a rotation matrix that is a function of absolute position
- Inside attention, rotate the queries and keys. When the queries and keys are multiplied (inner product), this results in an attention matrix that is a function of relative position only
- Mathematically, a relative position embedding $f(x, i)$ has the property

$$\langle f(x, i), f(y, j) \rangle = g(x, y, i - j)$$

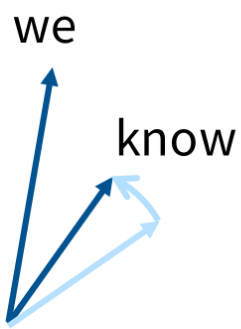
https://krasserm.github.io/2022/12/13/rotary-position-embedding/

RoPE: rotary position embeddings

- Inner products are invariant to arbitrary rotation: $\langle \mathrm{Rx}, \mathrm{Ry} \rangle = \langle x, y \rangle$
- So we can rotate each vector based on its absolute position, and the inner product will only reflect the relevant difference between the two positions

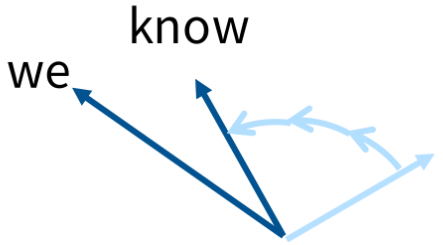


Position independent embedding



Embedding
"we know that"

Rotate we by '0 positions'
know by '1 positions'



Embedding
"of course we know"

Rotate we by '2 positions'
Rotate know by '3 positions'

RoPE: rotary position embeddings

- In 2D use a rotation matrix

$$\begin{pmatrix} cos\, m\theta & -sin\, m\theta \\ sin\, m\theta & cos\, m\theta \end{pmatrix}$$

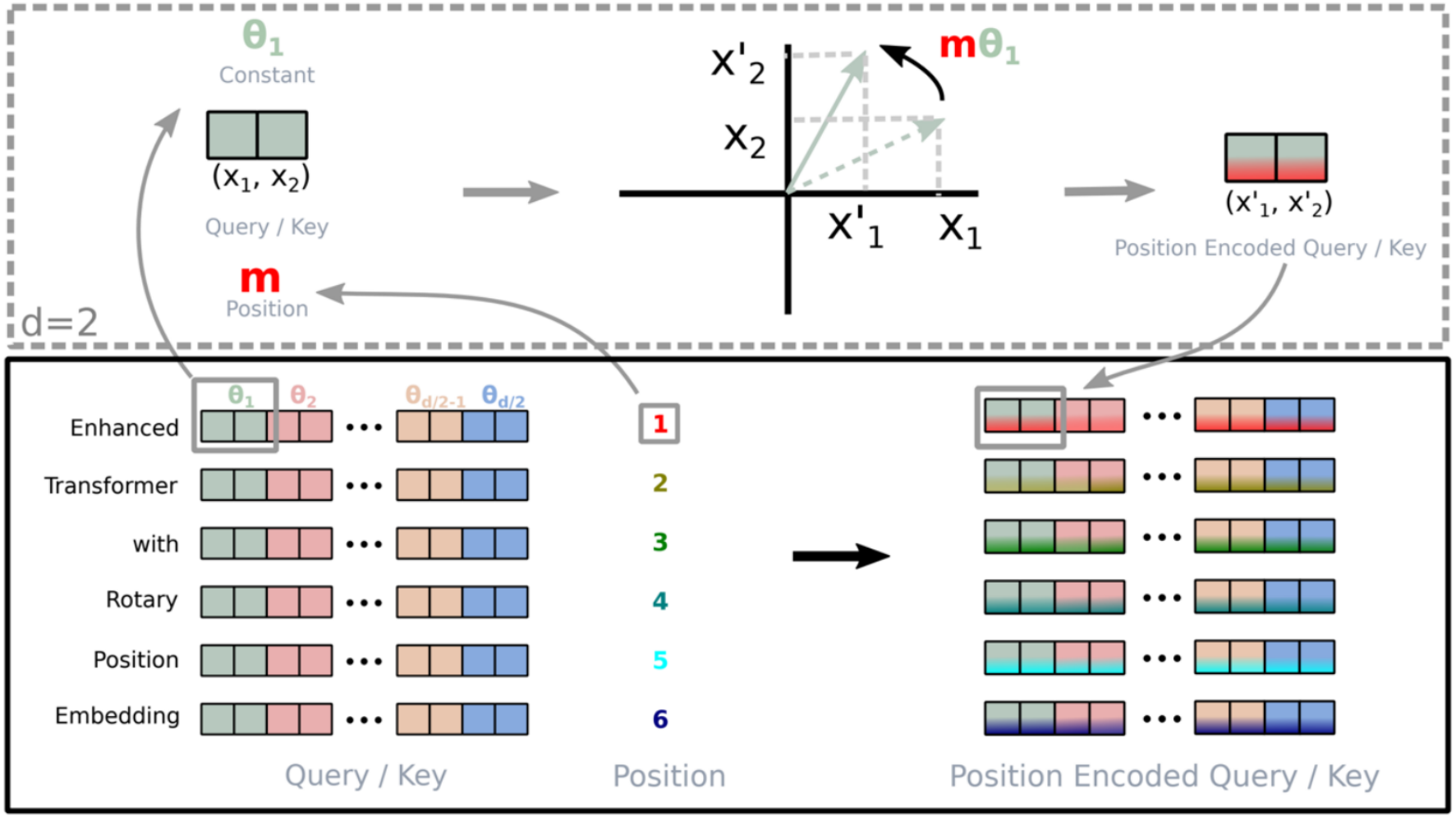
Where m is the absolute position and θ is a hyperparameter

- For a *d*-dimensional vector, take pairs of coordinates and rotate each in 2D

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^d_{\Theta,m} \boldsymbol{W}_{\{q,k\}} \boldsymbol{x}_m \qquad \theta_i = b^{-2i/d}$$

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

RoPE: rotary position embeddings

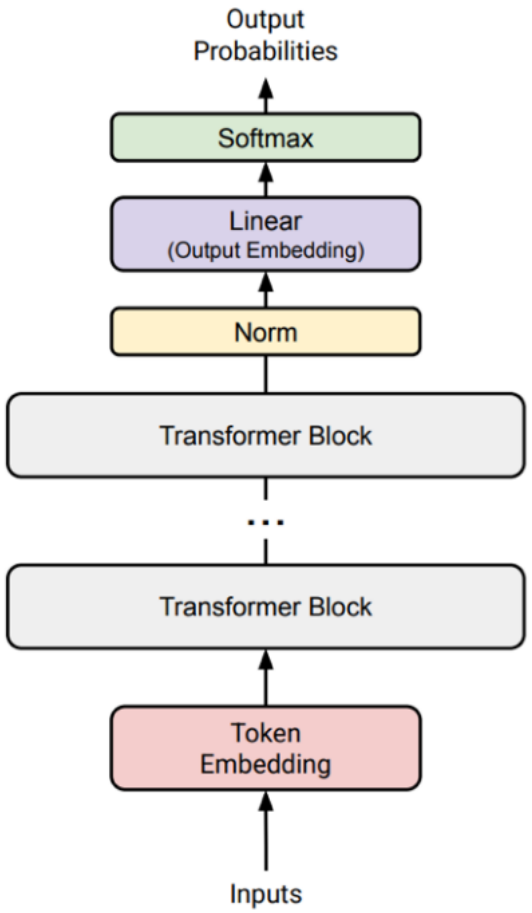


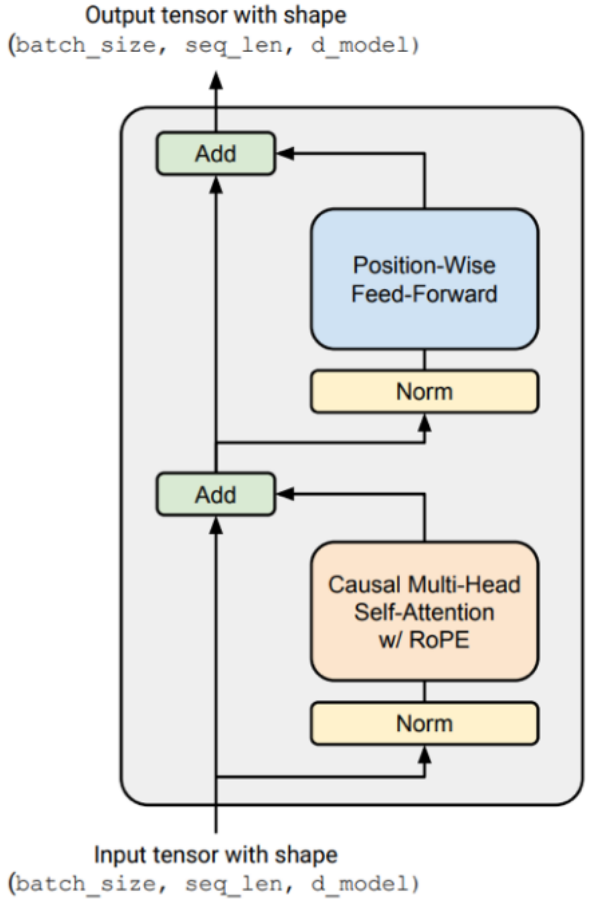
Su et al., 2021

RoPE: rotary position embeddings

- RoPE is applied to the queries and keys, not the values, before multi-head attention
- It is applied at each stacked layer, which reinforces positional information throughout the model

Transformer architecture





Modern variant:

- Grouped-Query Attention and KV cache for MHA
- Rotary position embeddings (RoPE)
- **FF layers using SwiGLU, no bias terms**
- LayerNorm (RMSNorm) before MHA/FF blocks

Plus particular hyperparameter choices

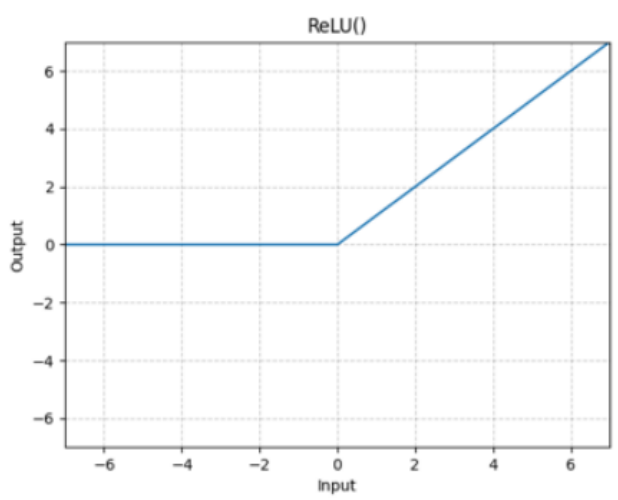
Feedforward layers

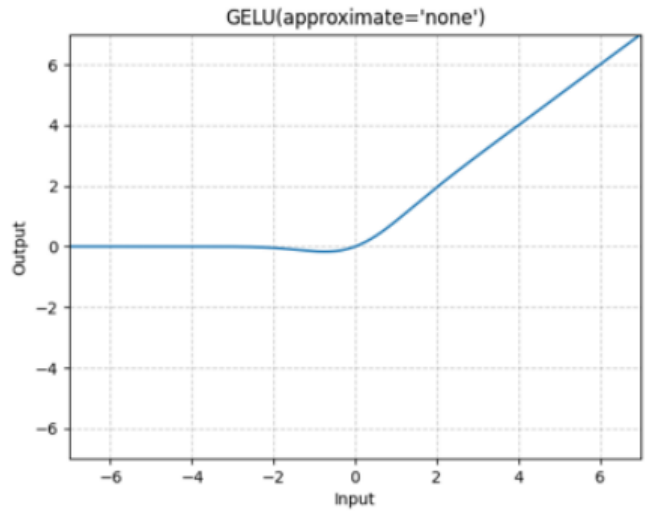
- Feedforward layers are applied at each position (sharing the same parameters)
- In large LLMs, feedforward layers actually dominate the computational cost (FLOPS) and number of parameters over the attention layers, despite their importance
- The original transformer feedforward network uses a rectified linear unit (ReLU) non-linearity

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Feedforward layers

- It has become standard practice to drop the bias terms
  - Saves memory lookups which improves performance
  - Improves training stability
- Various activation functions have been used in the FFN, including ReLU and GeLU





Gated activations

Most LLMs now use gated activations, which modifies the hidden layer computation with a *gate*

- FFN with ReLU (no bias terms):

$$FF(x) = \max(0, xW_1)\, W_2$$

- Using ReGLU - gated variant of ReLU:

$$\mathrm{FF}_{\mathrm{ReGLU}}(x) = (\max(0, xW_1) \otimes xV)\, W_2$$

- Note the extra parameter matrix *V*

Gated activations

- GeGLU (e.g. used in Gemma models)

$$\mathrm{FFN}_{\mathrm{GEGLU}}(x, W, V, W_2) = (\mathrm{GELU}(xW) \otimes xV)W_2$$

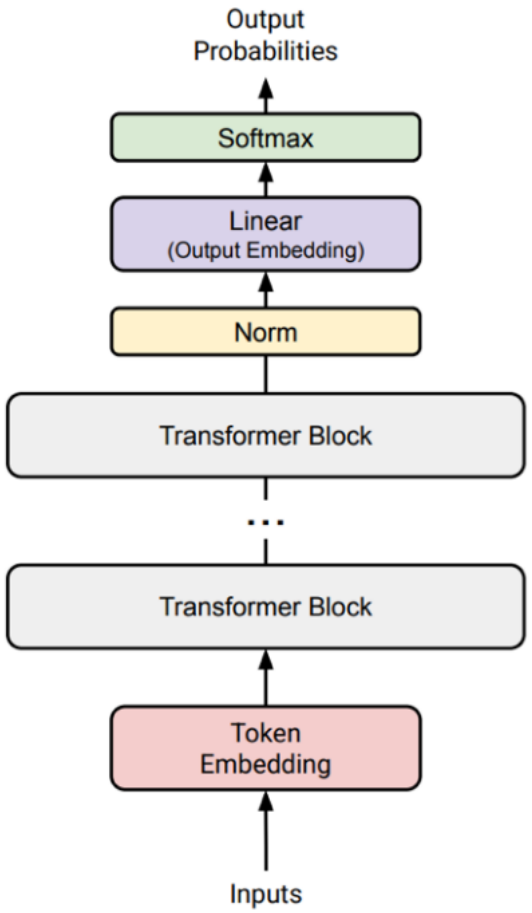
- SwiGLU (e.g. used in Llama models and other recent models)
  - Based on the swish activation: $\mathrm{swish}(x) = x\ \mathrm{sigmoid}(x)$

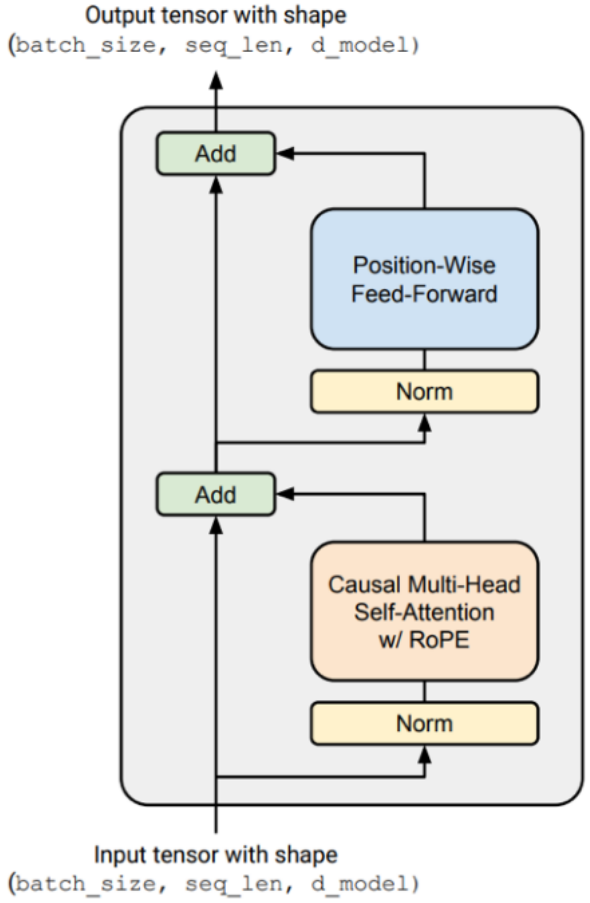
$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x, W, V, W_2) = (\mathrm{Swish}_1(xW) \otimes xV)W_2$$

Feedforward hyperparameters

- Standard practice for the relationship between the model dimension d and the feedforward hidden dimension d_{ff} is $d_{ff} = 4\,d$
- However FFNs with gated activation scale this down by 2/3 to compensate for having 3 parameter matrices instead of 2, resulting in $d_{ff} = \frac{8}{3}\,d$
- These are heuristics, not absolute rules

Transformer architecture





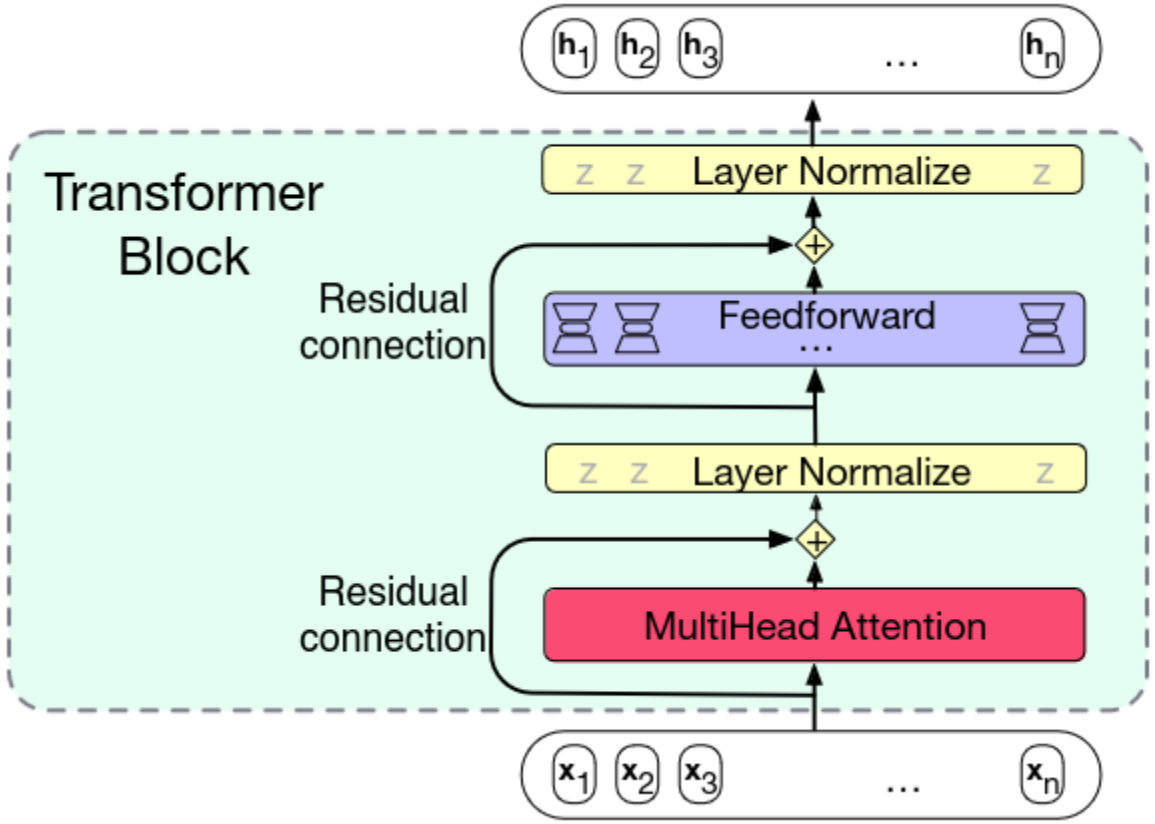
Modern variant:

- Grouped-Query Attention for MHA
- Rotary position embeddings (RoPE)
- FF layers using SwiGLU, no bias terms
- **LayerNorm (RMSNorm) before MHA/FF blocks**

Plus particular hyperparameter choices

Transformer block

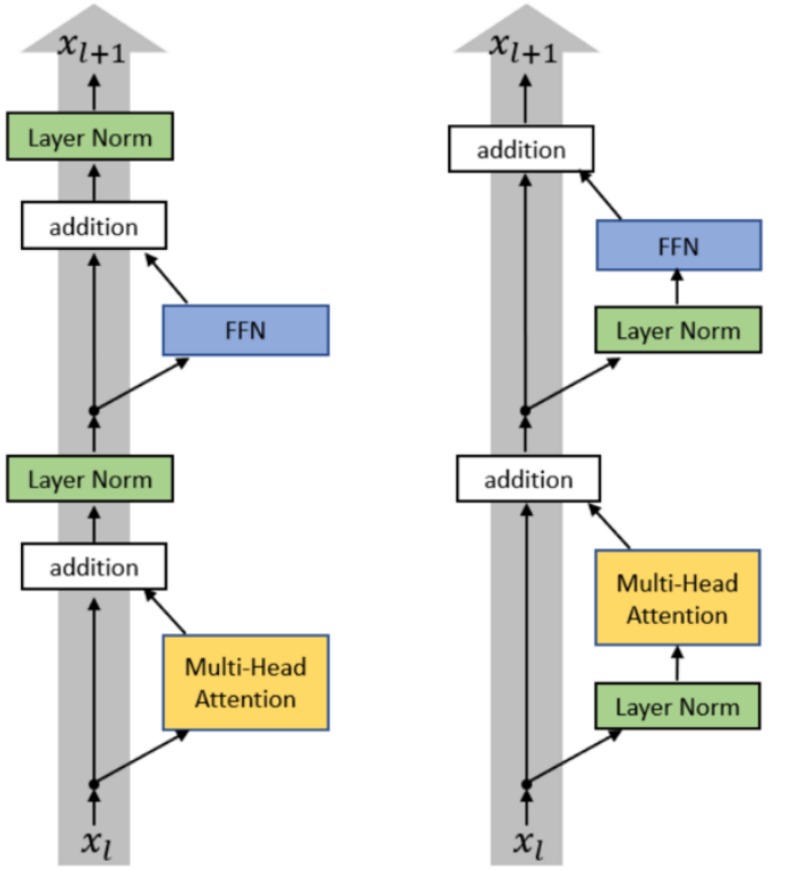
- The transformer block connects multi-head attention and the feedforward network with a residual connection and layer normalization
- However most LLMs change the order of these operations compared to the original Transformer



Jurafsky and Martin (2024) Ch. 10

Transformer block

- Layer normalization is performed *before* the MHA and FFN and outside the residual stream
- This is known as pre-norm
- This leads to more stable training for large networks



Original

Current approach

Figure from Xiong 2020

Transformer block

- Equations with pre-norm

Pre-LN Transformer

$$
\begin{aligned}
x_{l,i}^{pre,1} &= \text{LayerNorm}(x_{l,i}^{pre}) \\
x_{l,i}^{pre,2} &= \text{MultiHeadAtt}(x_{l,i}^{pre,1}, [x_{l,1}^{pre,1}, \cdots, x_{l,n}^{pre,1}]) \\
x_{l,i}^{pre,3} &= x_{l,i}^{pre} + x_{l,i}^{pre,2} \\
x_{l,i}^{pre,4} &= \text{LayerNorm}(x_{l,i}^{pre,3}) \\
x_{l,i}^{pre,5} &= \text{ReLU}(x_{l,i}^{pre,4} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l} \\
x_{l+1,i}^{pre} &= x_{l,i}^{pre,5} + x_{l,i}^{pre,3}
\end{aligned}
$$

Final LayerNorm: $x_{Final,i}^{pre} \leftarrow \text{LayerNorm}(x_{L+1,i}^{pre})$

Layer Norm

- The original layer norm normalizes the mean and variance across the vector dimension:

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

- **RMSNorm** does not subtract the mean or add a bias term:

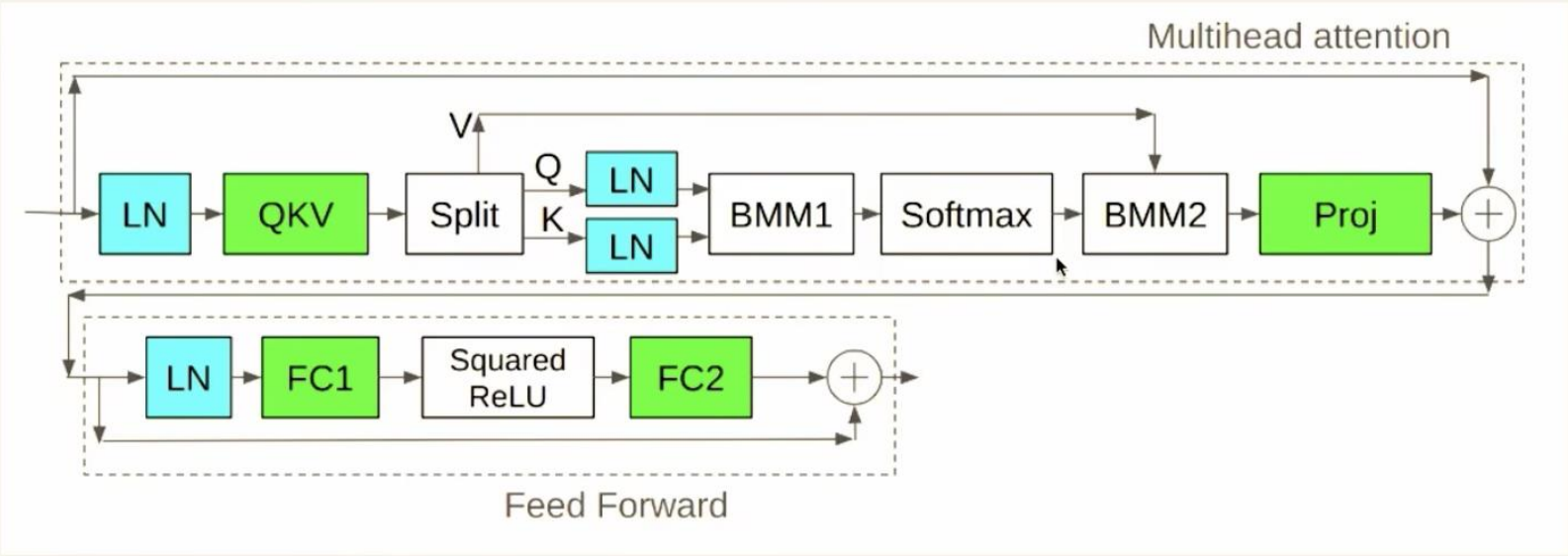
$$y = \frac{x}{\sqrt{||x||_2^2 + \varepsilon}} * \gamma$$

RMSNorm

- Most current LLMs use RMSNorm
- This is faster, in particular due to less data movement
- It has also been observed to lead to more stable training
- Similar principle to dropping bias terms from FFN

Additional LayerNorms

- Inserting more LayerNorms into the architecture can further improve training stability
- It is common to add layer norms before the queries and keys in attention (QK norm)



Other architecture options

- Sliding window attention – don’t attend to the full context at each layer
- Linear attention / state space models (included in hybrid architecture in large models)
- Mixture of Experts – more expressive, faster due to fewer active parameters during inference, more complex to train

References

Best original reference is tech reports of open-weight models

- Llama 3: https://arxiv.org/abs/2407.21783
- Qwen 2.5: https://arxiv.org/abs/2412.15115
- Gemma 2: https://arxiv.org/abs/2408.00118
- OLMo 2: https://arxiv.org/abs/2501.00656